SIMATS ENGINEERING

ASSESSMENT TOOL – C0-1 AT-2
COURSE NAME: Artificial Intelligence
COURSE CODE: CSA1735
NAME: Raghavan. K
Reg no: 192417003

Code of Conduct: I ____Raghavan. K/ 192417003__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Raghavan. K

Question 1 - Hospital duty scheduling as a constraint satisfaction problem

Aim

Assign one distinct shift to each doctor while meeting the stated restrictions: Doctor 1 cannot work Night; Doctor 2 must work earlier than Doctor 3; Doctor 3 cannot work Morning.

Algorithm

1. Create variables D1, D2, D3 with domains Morning, Afternoon, Night.
2. Enforce all-different shifts, D1 != Night, rank(D2) < rank(D3), and D3 != Morning.
3. Assign D2, then D3, then D1 using backtracking.
4. Reject a partial assignment as soon as a constraint is violated. Return the first complete consistent assignment.

Program

```
shifts = ["Morning", "Afternoon",
"Night"] rank = {"Morning": 0,
"Afternoon": 1, "Night": 2} order =
["D2", "D3", "D1"] assignment = {}

def consistent(a):     if
len(set(a.values())) != len(a):
        return False     if
a.get("D1") ==
"Night":
        return False     if "D2" in a
and "D3" in a:         if
rank[a["D2"]] &gt;=
rank[a["D3"]]:
            return False     if
a.get("D3") ==
"Morning":
        return False
return True

def solve(i=0):
    if i ==
len(order):
return True
doctor = order[i]
for shift in shifts:
```

```
        assignment[doctor] = shift         if
consistent(assignment) and solve(i +
1):
            return True
del
assignment[doctor]
return False

if solve():
    for doctor in ["D1", "D2", "D3"]:
        print(doctor, "=", assignment[doctor])
print("All constraints satisfied:",
consistent(assignment)) else:     print("No
solution")
```

Output

D1 = Afternoon
D2 = Morning
D3 = Night
All constraints satisfied: True D2 is earlier than D3, all

shifts are different, and D1 is not assigned Night.

Question 2 - Optimal path using an informed search

Assumed example: a 5 x 5 four-direction grid; start (1,1); goal (5,5); blocked cell (2,2). Coordinates are (row,column), numbered from 1. A* uses the Manhattan-distance heuristic. Greedy Best-First Search uses only h and is not guaranteed to return an optimal path, so A* is used to satisfy the optimal-path requirement.

Aim

Find a minimum-length route from the start to the goal in a grid with one blocked cell. The assessment does not specify a grid, start, goal, or obstacle locations, so this worked example states its assumptions.

Algorithm

1. Represent each free cell as a state; legal actions move one cell up, down, left, or right.
2. Set g(start)=0 and h(row,col)=|row-goal_row|+|col-goal_col|.
3. Put the start in a priority queue ordered by (f=g+h, h, row, column).
4. Repeatedly expand the lowest-priority cell and update each reachable neighbor if its g-cost improves.

5. Stop when the goal is removed from the queue, then follow parent pointers to reconstruct the path.

Program

```
import heapq

start, goal = (1, 1), (5, 5)
blocked = {(2, 2)}
moves = [(0, 1), (1, 0), (0, -1), (-1, 0)]

def h(p):
    return abs(p[0] - goal[0]) + abs(p[1] - goal[1])

def astar():
    queue = [(h(start), h(start), start[0],
start[1], 0)]     best = {start: 0}     parent =
{}     while queue:         f, estimate, r, c, g =
heapq.heappop(queue)         p = (r, c)
if g != best[p]:
            continue         if p ==
goal:             path = [p]
while p in parent:                 p =
parent[p]
path.append(p)             return
list(reversed(path)), g         for
dr, dc in moves:
            q = (r + dr, c + dc)             if not (1 &lt;=
q[0] &lt;= 5 and 1 &lt;= q[1] &lt;= 5):
                continue             if q in
blocked:                 continue
new_g = g + 1             if new_g &lt;
best.get(q, float("inf")):
                best[q] = new_g
parent[q] = p
                heapq.heappush(queue, (new_g +
h(q), h(q),                                     q[0], q[1],
new_g))     return None, float("inf")

path, cost = astar()
print("Path:", path)
print("Path length:",
cost)
```

Output

Assumed grid:
S . . . .
. # . . .
. . . . .
. . . . .
. . . . G

Path: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5)]
Path length: 8 moves
The Manhattan heuristic is admissible and consistent for four-direction unit-cost movement, so A* returns an optimal path.The Manhattan heuristic is admissible and consistent for four-direction unit-cost movement, so
A* returns an optimal path.

Question 3 - Rescue robot in a partially known environment

The assessment does not provide an actual map or risk values. The example below assumes a 4 x 4 grid, start (1,1), goal (4,4), blocked cells (2,2) and (3,2), and a risk cell (2,3). Entering a normal cell costs 1; entering the risk cell costs 3. A local sensor reports adjacent cells; the example route is calculated after the shown map has been revealed. The grid environment is discrete and sequential; it is partially observable before sensing and can be treated as deterministic for this example.

Aim

Model a rescue robot that must reach a victim while avoiding blocked and risky cells. Its rational action minimizes total travel and risk cost using sensor information and replanning.

Algorithm

1. Represent the robot state by its current grid cell and the set of cells learned by sensing.
2. Give safe moves cost 1 and a risky move its travel cost plus a risk penalty.
3. Sense adjacent cells, update the known map, and run Uniform Cost Search (Dijkstra's algorithm) on currently knownsafe cells.
4. Execute one planned move, sense again, and replan when new information changes the route.
5. Stop at the goal. A rational agent selects the action with the least expected cumulative cost subject to safetyconstraints.

Program

import heapq

```
start, goal = (1, 1), (4, 4)
blocked = {(2, 2), (3, 2)} risk =
(2, 3) moves = [(0, 1), (1, 0),
(0, -1), (-1, 0)]

def
least_cost_route()
:     queue = [(0,
start)]     best =
{start: 0}
parent = {}
while queue:
        cost, p =
heapq.heappop(queue)         if
cost != best[p]:
            continue         if
p == goal:             route
= [p]             while p in
parent:                 p =
parent[p]
route.append(p)
            return list(reversed(route)),
cost         for dr, dc in moves:
            q = (p[0] + dr, p[1] + dc)             if not (1
&lt;= q[0] &lt;= 4 and 1 &lt;= q[1] &lt;= 4):
                continue
if q in blocked:
                continue             step_cost = 3
if q == risk else 1             new_cost = cost
+ step_cost             if new_cost &lt;
best.get(q, float("inf")):
                best[q] = new_cost
parent[q] = p
                heapq.heappush(queue,
(new_cost, q))     return None, float("inf")

route, cost = least_cost_route()
print("Least-cost route:", route)
print("Total cost:", cost)
print("After each sensor update, run the planner again.")
```

Output

Assumed map:
S . . .
. # R .
. # . .
. . . G

Least-cost route: [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)]
Total cost: 6 The route avoids both blocked cells and the risk cell. Uniform Cost Search handles the non-uniform entry costs.

Question 4 - South India map coloring as a CSP

Adjacency used from the assessment map: TN-KL, TN-KA, TN-AP, KL-KA, KA-AP, KA-TG, KA-GA, KA-MH, AP-TG, GA-MH, and MH-TG. Variables are TN, KL, KA, AP, TG, GA, and MH.

Aim

Color the seven regions so adjacent regions have different colors, using Red, Green, and Blue. The problem is represented as a constraint satisfaction problem (CSP).

Algorithm

1. Give every region the domain {Red, Green, Blue}.
2. Connect adjacent regions with an inequality constraint.
3. Select the uncolored region with the smallest remaining domain (MRV); break ties by greater degree, thenalphabetically.
4. Try colors in Red, Green, Blue order and forward-check adjacent domains.
5. Backtrack if any uncolored region has no legal color.

Program

```
regions = ["TN", "KL", "KA", "AP",
"TG", "GA", "MH"] edges = [("TN",
"KL"), ("TN", "KA"), ("TN", "AP"),
("KL", "KA"), ("KA", "AP"), ("KA",
"TG"),
        ("KA", "GA"), ("KA", "MH"),
("AP", "TG"),          ("GA", "MH"),
("MH", "TG")] colors = ["Red",
"Green", "Blue"] neighbors = {r: set()
for r in regions} for a, b in edges:
   neighbors[a].add(b)
neighbors[b].add(a)
assignment = {}
```

```
def domain(region):
    used = {assignment[n] for n in
neighbors[region]            if n in
assignment}      return [color for color in
colors if color not in used]

def choose():
    remaining = [r for r in regions if r not in
assignment]      return min(remaining,
key=lambda r: (len(domain(r)),
-len(neighbors[r]), r))

def backtrack():      if
len(assignment) ==
len(regions):
        return True      r =
choose()      for color
in domain(r):
assignment[r] = color
        if all(domain(n) for n in neighbors[r]
if n not in assignment) and backtrack():
            return True
del assignment[r]
return False

backtrack() for r in
regions:      print(r, "=",
assignment[r])
```

Output

TN = Blue
KL = Green
KA = Red
AP = Green
TG = Blue
GA = Blue
MH = Green Every listed adjacent pair has different colors. The table/map

uses black text and white cells only.

Question 5 - 4 x 4 Sudoku using constraint propagation

Puzzle rows: - 2 - - / 3 - - 2 / - - 4 - / 4 - - 1. Each cell has domain {1,2,3,4}; every row, column, and 2 x 2 box must contain distinct values.

Aim

Solve the given 4 x 4 Sudoku by applying row, column, and 2 x 2 box constraints, using AC-3 propagation and backtracking only when propagation is insufficient.

Algorithm

1. Make each given cell a singleton domain and each blank cell domain {1,2,3,4}.
2. Build binary not-equal constraints between every pair of peer cells in the same row, column, or box.
3. Run AC-3: remove a value when it has no supporting different value in a peer's domain; enqueue affected arcs.
4. If domains are not all singletons, choose a smallest non-singleton domain, try each value, and recurse.
5. Display the completed grid.

Program

```
from copy import deepcopy

puzzle = [
[0, 2, 0, 0],
    [3, 0, 0, 2],
    [0, 0, 4, 0],
    [4, 0, 0, 1], ] cells = [(r, c) for r in
range(4) for c in range(4)]

def peers(x):     r, c = x     box_r,
box_c = (r // 2) * 2, (c // 2) * 2
return {y for y in cells if y != x and
(y[0] == r or y[1] == c or
(box_r &amp;lt;= y[0] &amp;lt; box_r + 2 and
box_c &amp;lt;= y[1] &amp;lt; box_c + 2))}

def ac3(domains):
    queue = [(x, y) for x in cells for y in
peers(x)]     while queue:
        x, y = queue.pop()         revised = [a
for a in domains[x]                  if any(a
!= b for b in domains[y])]         if revised
!= domains[x]:             domains[x] =
```

```
revised            if not revised:
return False
          queue.extend((z, x) for z in peers(x) if z !=
y)     return True

domains = {}
for r, c in
cells:
   value = puzzle[r][c]
   domains[(r, c)] = [value] if value else [1, 2, 3, 4]

def
search(current):
if not
ac3(current):
       return None     if
all(len(current[x]) == 1 for x in
cells):
       return current
   x = min((x for x in cells if
len(current[x]) &gt; 1),
key=lambda x: len(current[x]))     for value
in current[x]:        trial =
deepcopy(current)        trial[x] = [value]
answer = search(trial)        if answer is not
None:
          return answer
return None

solution =
search(domains) for r
in range(4):
   print(" ".join(str(solution[(r,
c)][0])                for c in
range(4)))
```

Output

1 2 3 4
3 4 1 2
2 1 4 3
4 3 2 1 All rows, columns, and 2 x 2 boxes contain 1,

2, 3, and 4 exactly once.

Question 6 - Cryptarithm TWO + TWO = FOUR

Aim

Assign a distinct decimal digit to each letter so that TWO + TWO = FOUR is arithmetically correct. T and F cannot be zero.

Algorithm

1. Create variables T, W, O, F, U, R with digits 0 through 9 and an all-different constraint.
2. Work from the units column to the thousands column, carrying 0 or 1 as needed.
3. Reject any assignment that violates a column equation or gives T or F the value zero.
4. Verify the complete addition and report the first valid solution.

Program

```
from itertools import permutations

letters = "TWOFUR" for T, W, O, F,
U, R in permutations(range(10), 6):
if T == 0 or F == 0:
        continue      units = 2 * O
if units % 10 != R:
continue      c1 = units // 10
tens = 2 * W + c1      if tens %
10 != U:          continue      c2 =
tens // 10      hundreds = 2 * T +
c2      if hundreds % 10 != O:
continue      c3 = hundreds // 10
if c3 != F:          continue      two
= 100 * T + 10 * W + O      four
= 1000 * F + 100 * O + 10 * U
+ R      if two + two == four:
        answer = dict(zip(letters, (T, W, O,
F, U, R)))          print(answer)
print(two, "+", two, "=", four)
print("Carries C1,C2,C3 =", c1, c2, c3)
break
```

Output

```
{'T': 7, 'W': 3, 'O': 4, 'F': 1, 'U': 6, 'R': 8}
734 + 734 = 1468
```

Carries C1,C2,C3 = 0 0 1 Column checks: 2O=8; 2W+C1=6;

2T+C2=14, giving O=4 and carry 1; the leading carry is F=1.

Question 7 - Ambulance route planning with A* and a flooded road

Undirected road costs: S-A=3, S-B=2, A-C=4, B-C=1, B-D=2, C-D=2, C-E=3, D-G=2, E-G=4. Heuristics: h(S)=6, h(A)=6, h(B)=4, h(C)=4, h(D)=2, h(E)=3, h(G)=0. The heuristic is admissible and consistent on these edges, including B-D.

Aim

Find an efficient route from S to hospital G using A*. The road B-D may be closed by flooding; compare the route with the road open.

Algorithm

1. Store the road graph, nonnegative edge costs, and heuristic values.
2. Keep OPEN ordered by f(n)=g(n)+h(n); keep the best known g-cost and parent for each node.
3. Skip a road when it is in the flooded-edge set. Relax other neighbors and update their parents.
4. Stop when G is selected from OPEN; reconstruct its route and cost.
5. Run once with B-D closed and once with all roads open.

Program

```
import heapq from
itertools import
count

edges = [("S", "A", 3), ("S", "B", 2),
("A", "C", 4),          ("B", "C", 1), ("B",
"D", 2), ("C", "D", 2),          ("C", "E",
3), ("D", "G", 2), ("E", "G", 4)] h =
{"S": 6, "A": 6, "B": 4, "C": 4,      "D":
2, "E": 3, "G": 0} graph = {n: [] for n in
h} for a, b, cost in edges:
    graph[a].append((b, cost))
graph[b].append((a, cost))

def astar(closed_roads):
    queue, serial = [], count()
heapq.heappush(queue, (h["S"],
next(serial), "S", 0))      best, parent = {"S":
0}, {}      expanded = []      while queue:
```

```
        f, _, node, g =
heapq.heappop(queue)        if g !=
best[node]:             continue
expanded.append(node)         if node
== "G":             path = [node]
while node in parent:                 node =
parent[node]
path.append(node)             return
list(reversed(path)), g, expanded
for nxt, cost in graph[node]:
            if frozenset((node, nxt)) in closed_roads:
                continue              new_g = g +
cost              if new_g &lt; best.get(nxt,
float("inf")):
                best[nxt] = new_g
parent[nxt] = node
                heapq.heappush(queue, (new_g + h[nxt],
next(serial), nxt, new_g))      return None,
float("inf"), expanded

flooded = {frozenset(("B",
"D"))} print("B-D closed:",
astar(flooded)) print("B-D
open:", astar(set()))
```

Output

With B-D closed:
OPEN/CLOSED trace:

1. Expand S: OPEN B(g=2,h=4,f=6), A(g=3,h=6,f=9); CLOSED {S}
2. Expand B: OPEN C(g=3,h=4,f=7), A(g=3,h=6,f=9); CLOSED {S,B}
3. Expand C: OPEN D(g=5,h=2,f=7), E(g=6,h=3,f=9), A(g=3,h=6,f=9); CLOSED {S,B,C}
4. Expand D: OPEN G(g=7,h=0,f=7), E(g=6,h=3,f=9), A(g=3,h=6,f=9); CLOSED {S,B,C,D}
5. Select G: route S -> B -> C -> D -> G; cost 7.

Expanded: ['S', 'B', 'C', 'D', 'G']

With B-D open: route S -> B -> D -> G; cost 6.
For every edge, |h(u)-h(v)| is at most its cost; thus the heuristic is consistent. The listed values also do not exceed shortesFor every edge, |h(u)-h(v)| is at most its cost; thus the heuristic is consistent. The listed values also do not exceed shortest remaining costs.

Question 8 - 4-Queens using Min-Conflicts

Initial assignment by column is rows [1,3,4,2]. On a tie, select the leftmost conflicted column and then the lowest-numbered row with minimum conflicts.

Aim

Place four queens on a 4 x 4 board so that no two queens share a row, column, or diagonal. Use Min-Conflicts with deterministic tie-breaking for a reproducible example.

Algorithm

1. Represent one queen per column by an array of row numbers.
2. Count row and diagonal attacks for each queen.
3. If conflicts remain, select the conflicted queen with the most conflicts; break ties by lowest column.
4. Move it to a row that minimizes its conflicts; break ties by lowest row.
5. Repeat until no conflicts remain or the step limit is reached.

Program

```
def conflicts(col, row, rows):
    total = 0     for other,
other_row in enumerate(rows):
if other == col:
            continue         if row == other_row or abs(row
- other_row) == abs(col - other):
            total += 1
return total

def min_conflicts(rows, limit=20):
    history =
[rows[:]]     for _ in
range(limit):
        counts = [conflicts(c, rows[c], rows) for c
in range(4)]         bad = [c for c in range(4) if
counts[c] &amp;gt; 0]         if not bad:
            return rows, history         col = min(bad,
key=lambda c: (-counts[c], c))         costs =
[conflicts(col, row, rows) for row in range(1, 5)]
best_cost = min(costs)         row = next(r for r in
range(1, 5)                   if conflicts(col, r, rows)
== best_cost)         rows[col] = row
history.append(rows[:])     return rows, history
```

```
solution, history = min_conflicts([1,
3, 4, 2]) for state in history:
    print(state, "conflicts =", sum(
        conflicts(c, state[c], state) for c in range(4)) // 2)
print("Solution:", solution)
```

Output

[1, 3, 4, 2] conflicts = 1
[1, 1, 4, 2] conflicts = 1
[3, 1, 4, 2] conflicts = 0 Final rows by column: Q1=3, Q2=1, Q3=4, Q4=2. Each

column contains one queen and no queens attack each other.

Question 9 - Missionaries and Cannibals using BFS

Aim

Move three missionaries and three cannibals across a river using a two-person boat. Cannibals must never outnumber missionaries on either bank where missionaries are present.

Algorithm

1. Represent a state as (missionaries_left, cannibals_left, boat_side).
2. Generate boat loads of one or two people that are available on the boat's bank.
3. Reject any successor where cannibals outnumber missionaries on either bank with missionaries present.
4. Use Breadth-First Search with a visited set and parent links.
5. Reconstruct the first goal path; BFS minimizes the number of crossings.

Program

```
from collections import deque

loads = [(0, 2, "CC"), (0, 1, "C"), (2,
0, "MM"),          (1, 0, "M"), (1, 1,
"MC")] start, goal = (3, 3, "L"), (0,
0, "R")

def safe(m, c):
return m == 0 or m
&gt;= c

def successors(state):     ml, cl, boat
= state     result = []     for dm, dc,
label in loads:          if boat == "L":
```

```
if dm &gt; ml or dc &gt; cl:
continue            nxt = (ml - dm, cl -
dc, "R")        else:            if dm
&gt; 3 - ml or dc &gt; 3 - cl:
            continue            nxt = (ml + dm, cl +
dc, "L")        if safe(nxt[0], nxt[1]) and safe(3 -
nxt[0], 3 - nxt[1]):
          result.append((nxt, label))
return result

queue =
deque([start])
parent = {start:
None}
move_used = {}
while queue:
   state =
queue.popleft()     if
state == goal:
       break     for nxt, label in
successors(state):        if nxt
not in parent:
parent[nxt] = state
move_used[nxt] = label
queue.append(nxt)

path = [] state = goal while state is not
None:     path.append((state,
move_used.get(state, "Start")))     state =
parent[state] path.reverse() for i, (state,
move) in enumerate(path):     print(i,
move, state)
```

Output

Crossing trace (boat passengers and resulting state (M_left,C_left,boat)):

```
0 Start (3,3,L)
1 CC -&gt; (3,1,R)
2 C &lt;- (3,2,L)
3 CC -&gt; (3,0,R)
4 C &lt;- (3,1,L)
5 MM -&gt; (1,1,R)
6 MC &lt;- (2,2,L)
```

MM -> (0,2,R)
C <- (0,3,L) 9 CC -> (0,1,R)
C <- (0,2,L)
CC -> (0,0,R) The solution uses 11 crossings and all intermediate

states are safe.

Question 10 - Sensorless corridor using belief states

Aim

Guide an agent to the end of a four-cell corridor without knowing its starting cell. Compare belief-state planning with a right-wall bump sensor.

Algorithm

1. Represent uncertainty as a belief set of possible cells, initially {1,2,3,4}.
2. For action Right, move each possible position one cell right, stopping at cell 4.
3. Apply Right repeatedly and update the entire belief set after each action.
4. Stop when the belief becomes the singleton {4}.
5. With a bump sensor, repeat Right until a bump confirms the wall at cell 4; compare the worst-case action count.

Program

```
belief = {1, 2, 3, 4}

def move_right(states):
    return {min(4, position + 1) for position in states}

for step in range(1, 4):
    belief = move_right(belief)
print("After Right", step, ":",
sorted(belief))

def bump_policy(start):
    position, actions = start,
0     while True:
        actions += 1
if position == 4:
            return 4, actions
position += 1

print("Bump policy (start, final cell,
actions):") for start in range(1, 5):
print(start, bump_policy(start))
```

Output

Sensorless belief updates:
After Right 1: [2, 3, 4]
After Right 2: [3, 4]
After Right 3: [4]

Three Right actions guarantee arrival at cell 4 from any starting cell. This is minimal because starting at cell 1 requires thrThree Right actions guarantee arrival at cell 4 from any starting cell. This is minimal because starting at cell 1 requires three moves. With a bump sensor, repeating Right until the wall is detected takes 1 to 4 actions including the bump; it confirms arrival while giving feedback.